



MÓDULO DIW · DAW — UNIDAD DIDÁCTICA 2

# Penpot Avanzado, Design Systems y Design Tokens

RA1 · Guía de estilo, marcos/capas y plantillas de diseño

- Diseñar componentes con variantes e instancias en Penpot
- Construir una jerarquía de tokens de diseño en 3 niveles
- Documentar una guía de estilo y plantillas de diseño reutilizables



## CONTEXTO PROFESIONAL

# ¿Por qué esto importa en el mundo real?

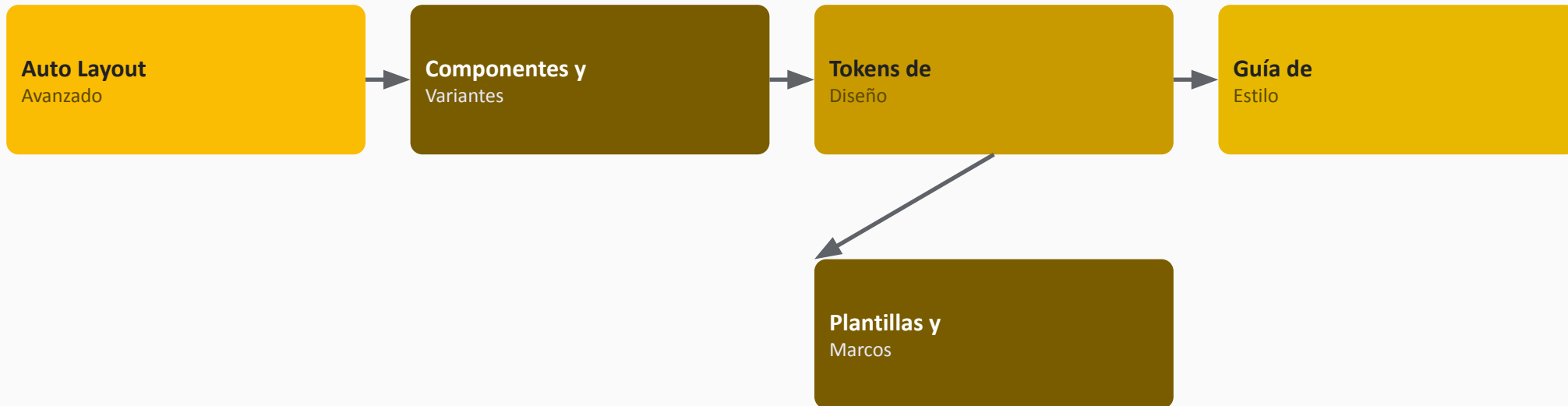
- **Consistencia visual a escala:** un design system evita que cada página se vea 'ligeramente distinta'
- **Puesto de trabajo real:** mantener un sistema de diseño coherente es tarea diaria en cualquier equipo de producto
- **Adopción de la industria:** Google Material, Apple HIG, Atlassian son ejemplos de design systems públicos de referencia
- **Impacto de negocio:** un design system reduce el tiempo de desarrollo de nuevas pantallas drásticamente
- **Escalabilidad del equipo:** nuevos desarrolladores son productivos antes si el sistema está bien documentado



**DATO CLAVE** — Empresas con design systems maduros (Google, Airbnb, Atlassian) publican sus sistemas como referencia abierta para toda la industria.



# Índice y mapa conceptual de la unidad



**CLAVE** — Hoja de ruta de hoy: de Auto Layout avanzado a un mini Design System documentado con tokens en 3 niveles.



# De Penpot a CSS: el Pipeline de Tokens

tokens.css

```
:root {  
  --blue-500: #4285F4;  
  --color-brand-primary: var(--blue-500);  
  --button-bg-primary: var(--color-brand-primary);  
}  
  
.boton {  
  background: var(--button-bg-primary);  
}
```



**NOTA** — Este es el pipeline nativo de Penpot: Design Tokens (panel Tokens) → tokens.json en formato W3C DTCG → CSS custom properties, con 3 niveles (global, semántico, componente).



# Los Tres Niveles de un Token



## Nivel 1: Global

Paleta bruta, sin significado de uso (ej. blue-500)



## Nivel 2: Semántico

Da significado al valor bruto (ej. color-brand-primary)



## Nivel 3: Componente

Ligado a un componente concreto (ej. button-bg-primary)



## Saltarse el nivel semántico

Usar blue-500 directo en componentes — error habitual a evitar

## Regla de oro

Si usas un valor global (blue-500) directamente en un componente, sin pasar por el nivel semántico, pierdes la razón de ser del sistema de tokens.



# Un Token, Tres Representaciones

Mismo token, distinto formato

```
Penpot Token:  color/brand/primary  
CSS:          --color-brand-primary  
JSON:         "color.brand.primary": "#1E88E5"  
Tailwind:     colors.brand.primary
```



**NOTA** — El mismo valor de token se expresa con sintaxis distinta según la herramienta, pero siempre representa la misma decisión de diseño.



# Sistematizando Colores y Espaciados

design-tokens.css

```
:root {  
  --color-texto: #202124;  
  --color-fondo: #FFFFFF;  
  --espaciado-sm: 8px;  
  --espaciado-md: 16px;  
  --espaciado-lg: 32px;  
}
```



**NOTA** — Un token de diseño no es solo una variable — es una decisión de diseño documentada y reutilizable en todo el proyecto.



# Cambiando Todo el Tema con una Variable

theme.css

```
[data-theme="dark"] {  
  --color-texto: #E8EAED;  
  --color-fondo: #202124;  
}
```



**NOTA** — Al reasignar las mismas variables bajo un atributo [data-theme], todo el sitio cambia de tema sin tocar ni una sola regla más.



# Plantillas de Diseño Reutilizables



## Plantilla de página

Layout de listado con header, filtros y grid ya resuelto



## Plantilla de componente

Ficha de tarjeta con placeholders reutilizables



## Plantilla de flujo

Checkout de 3 pasos con la navegación ya definida



## Diseñar sin plantillas

Cada pantalla nueva se construye desde cero, sin reutilización

## Idea clave

Una plantilla no es una limitación creativa: es la forma de no repetir decisiones ya resueltas en cada pantalla nueva.



# Nomenclatura de Capas en Penpot

estructura-capas

```
Card (Frame)
├ Title (Text)
└ Image (Image)
Card/Featured (Variant)
```



**NOTA** — Nombrar las capas de Penpot con la misma disciplina que el código facilita muchísimo la traducción a HTML/CSS en UD3-UD5.



# Construir un Mini Design System

1

## Definir tokens

8 tokens de color (3 niveles) y 5 de espaciado

2

## Crear componente Button

Con 3 variantes: Primary, Secondary, Disabled

3

## Crear componente Card

Que use 2 instancias de Button

4

## Documentar guía de estilo

Paleta, tipografía, componentes y reglas de uso

5

## Exportar tokens

Con el plugin Tokens Studio a tokens.json



**ÉXITO** — Criterio de éxito: los 3 niveles de tokens están claramente diferenciados y Card reutiliza instancias de Button sin duplicar estilos.



## CASO REAL

# Design Systems en la Industria

- **Material Design (Google):** sistema de diseño completo, documentado y de código abierto
- **Componentes como Storybook:** catálogo visual interactivo de todos los componentes del sistema
- **Design tokens multiplataforma:** los mismos tokens alimentan web, iOS y Android desde una única fuente
- **Versionado del sistema:** un design system evoluciona igual que cualquier software, con versiones
- **Colaboración diseño-desarrollo:** el design system es el lenguaje común entre diseñadores y programadores



# Trampas de Especificidad y Cascada

## **Saltarse el nivel semántico**

Usar blue-500 directo en 40 componentes en vez de un alias semántico

## **No usar Auto Layout**

Frames posicionados a mano se rompen con cualquier cambio de contenido

## **Variantes sin Boolean/Instance Swap**

Duplicar componentes completos en vez de usar propiedades de variante

## **Colores sin token asociado**

Cambiar la marca implica buscar y reemplazar en decenas de frames



# Trampas de Organización

## Nombrar capas sin convención

Cada diseñador nombra los boards a su manera, generando caos

## Tokens no documentados

Nadie sabe qué variable usar para qué propósito

## Duplicar componentes por tema

En vez de reasignar tokens, se duplica el componente completo

## Guía de estilo sin actualizar

El sistema se desincroniza entre Penpot y código en pocas semanas



## Preguntas de Control y Debate

1 ¿Por qué es un error saltarse el nivel semántico y usar el valor global directamente?

2 ¿Qué diferencia hay entre una instancia de componente y un override puntual?

3 ¿Qué es exactamente un token de diseño, más allá de 'una variable'?

4 ¿Cómo permite un [data-theme] cambiar todo el tema con una sola reasignación?

5 ¿Por qué una plantilla de diseño acelera el trabajo sin limitar la creatividad?



## Reto Express (15-20 min, en parejas)

1

### Definir 5 tokens

Primario, secundario, texto, fondo y espaciado

2

### Crear un componente con variantes

Al menos 2 variantes visuales distintas

3

### Crear una plantilla

De página reutilizable con placeholders

4

### Verificar los 3 niveles

Que ningún componente use un valor global directo

5

### Exportar tokens nativos

Generar el archivo tokens.json



**ÉXITO** — Criterio de éxito: los tokens exportados reflejan exactamente los 3 niveles definidos en Penpot, sin saltos de nivel.



# Resizing de Flex Layout, en CSS

layout.css

```
.contenedor {  
  width: fit-content; /* Fit */  
}
```



**NOTA** — Fit, Fill (100%) y Fix en Penpot tienen equivalencia directa en CSS: fit-content, flex:1 y un ancho fijo.



## Exportando Tokens desde Penpot

- **Design tokens desde diseño:** el panel Tokens de Penpot exporta colores y espaciados directamente en formato W3C DTSG (JSON)
- **Fuente única de verdad:** diseñadores y desarrolladores comparten exactamente los mismos valores
- **Automatización del flujo:** cambios en Penpot pueden regenerar automáticamente el fichero de tokens
- **Reduce errores de transcripción:** copiar valores hexadecimales a mano es una fuente histórica de inconsistencias



ÚLTIMA REFLEXIÓN

## Una Pregunta Más Antes del Resumen

1

¿Qué ventaja de mantenimiento aporta un design system frente a estilos ad-hoc por página?

# Resumen de Puntos Clave

- ✓ Los tokens se organizan en 3 niveles: global, semántico y de componente
- ✓ Saltarse el nivel semántico rompe la razón de ser del sistema de tokens
- ✓ Un token de diseño documenta una decisión, no es solo una variable más
- ✓ El modo oscuro bien implementado reasigna variables, no duplica reglas
- ✓ Con RA1 completo, el bloque de diseño visual termina — a partir de UD3 escribimos código

➔ **SIGUIENTE** — UD3 — HTML5 Semántico, Estructura Accesible y Formularios